

✅ Checklist complète - Projet Truffière

📁 Fichiers créés pour vous

Fichiers racine

- ✅ `(docker-compose.yml)` - Configuration Docker Compose
- ✅ `(init-db.sql)` - Schéma de base de données PostgreSQL
- ✅ `(README.md)` - Documentation principale
- ✅ `(INSTRUCTIONS_DEPLOYMENT.md)` - Guide de déploiement détaillé
- ✅ `(QUICK_REFERENCE.md)` - Aide-mémoire des commandes
- ✅ `(CUSTOMIZATION.md)` - Guide de personnalisation
- ✅ `(CHECKLIST.md)` - Ce fichier
- ✅ `(setup.sh)` - Script d'installation automatique (Linux/Mac)
- ⚠️ `(.gitignore)` - À créer (fourni dans setup.sh)

Backend (`(backend/)`)

- ✅ `(Dockerfile)` - Configuration Docker backend
- ✅ `(package.json)` - Dépendances npm
- ✅ `(server.js)` - Serveur Express avec toutes les routes API
- ⚠️ `(.env)` - À créer (fourni dans setup.sh)

Frontend (`(frontend/)`)

-  `Dockerfile` - Configuration Docker frontend
-  `package.json` - Dépendances React
-  `public/index.html` - HTML principal (fourni dans setup.sh)
-  `src/index.js` - Point d'entrée React (fourni dans setup.sh)
-  `src/App.js` - Composant principal avec navigation
-  `src/App.css` - Styles de l'application
-  `src/components/Dashboard.js` - Tableau de bord
-  `src/components/Parcelles.js` - Gestion des parcelles avec CRUD
-  `src/components/Arbres.js` - À créer séparément
-  `src/components/Interventions.js` - À créer séparément
-  `src/components/Recoltes.js` - À créer séparément
-  `src/components/Clients.js` - À créer séparément
-  `src/components/Ventes.js` - À créer séparément
-  `src/components/Statistiques.js` - À créer séparément

Nginx (optionnel pour production)

-  `nginx/nginx.conf` - Configuration Nginx

Actions à effectuer

1. Créer la structure de dossiers

```
bash

mkdir -p truffiere/backend
mkdir -p truffiere/frontend/public
mkdir -p truffiere/frontend/src/components
mkdir -p truffiere/nginx
cd truffiere
```

2. Copier les fichiers fournis

Fichiers racine

- ☐ Copier `docker-compose.yml`
- ☐ Copier `init-db.sql`
- ☐ Copier `README.md`
- ☐ Copier `INSTRUCTIONS_DEPLOYMENT.md`
- ☐ Copier `QUICK_REFERENCE.md`
- ☐ Copier `CUSTOMIZATION.md`
- ☐ Copier `CHECKLIST.md`
- ☐ Copier `setup.sh` (optionnel)

Backend

- ☐ Copier `backend/Dockerfile`
- ☐ Copier `backend/package.json`
- ☐ Copier `backend/server.js`

Frontend - Fichiers principaux

- ☐ Copier `frontend/Dockerfile`
- ☐ Copier `frontend/package.json`
- ☐ Copier `frontend/src/App.js`
- ☐ Copier `frontend/src/App.css`

Frontend - Composants

- ☐ Copier `frontend/src/components/Dashboard.js`
- ☐ Copier `frontend/src/components/Parcelles.js`

3. ⚠ IMPORTANT : Séparer les composants React

Le fichier "**Composants React restants**" que je vous ai fourni contient 6 composants en un seul fichier. Vous DEVEZ les séparer :

Créer ces fichiers en copiant chaque section :

`frontend/src/components/Arbres.js`

javascript

```
import React, { useState, useEffect } from 'react';
import axios from 'axios';

const API_URL = process.env.REACT_APP_API_URL || 'http://localhost:3001/api';

function Arbres() {
  // Copier le code de la section Arbres
}

export default Arbres;
```

frontend/src/components/Interventions.js

```
javascript

import React, { useState, useEffect } from 'react';
import axios from 'axios';

const API_URL = process.env.REACT_APP_API_URL || 'http://localhost:3001/api';

function Interventions() {
  // Copier le code de la section Interventions
}

export default Interventions;
```

frontend/src/components/Recoltes.js

```
javascript

import React, { useState, useEffect } from 'react';
import axios from 'axios';

const API_URL = process.env.REACT_APP_API_URL || 'http://localhost:3001/api';

function Recoltes() {
  // Copier le code de la section Recoltes
}

export default Recoltes;
```

frontend/src/components/Clients.js

```
javascript

import React, { useState, useEffect } from 'react';
import axios from 'axios';

const API_URL = process.env.REACT_APP_API_URL || 'http://localhost:3001/api';

function Clients() {
  // Copier le code de la section Clients
}

export default Clients;
```

frontend/src/components/Ventes.js

javascript

```
import React, { useState, useEffect } from 'react';
import axios from 'axios';

const API_URL = process.env.REACT_APP_API_URL || 'http://localhost:3001/api';

function Ventes() {
  // Copier le code de la section Ventes
}

export default Ventes;
```

frontend/src/components/Statistiques.js

javascript

```
import React, { useState, useEffect } from 'react';
import axios from 'axios';

const API_URL = process.env.REACT_APP_API_URL || 'http://localhost:3001/api';

function Statistiques() {
  // Copier le code de la section Statistiques
}

export default Statistiques;
```

4. Créer les fichiers générés automatiquement

Si vous utilisez le script `setup.sh`, ces fichiers seront créés automatiquement. Sinon, créez-les manuellement :

frontend/public/index.html

html

```
<!DOCTYPE html>
<html lang="fr">
  <head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <title>Gestion de Truffière</title>
    <link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css" />
  </head>
  <body>
    <noscript>Vous devez activer JavaScript pour utiliser cette application.</noscript>
    <div id="root"></div>
  </body>
</html>
```

frontend/src/index.js

```
javascript

import React from 'react';
import ReactDOM from 'react-dom/client';
import './App.css';
import App from './App';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

backend/.env

```
env

NODE_ENV=development
PORT=3001
DB_HOST=postgres
DB_PORT=5432
DB_NAME=truffiere
DB_USER=truffiere_user
DB_PASSWORD=truffiere_pass_2024
```

.gitignore (racine)

```
node_modules/
.env
*.log
.DS_Store
```

5. Vérifier la structure finale

Votre arborescence doit ressembler à ceci :

```
truffiere/
├── .gitignore
├── docker-compose.yml
├── init-db.sql
├── README.md
├── INSTRUCTIONS_DEPLOYMENT.md
├── QUICK_REFERENCE.md
├── CUSTOMIZATION.md
├── CHECKLIST.md
├── setup.sh (optionnel)
├── backend/
│   ├── .env
│   ├── Dockerfile
│   ├── package.json
│   └── server.js
├── frontend/
│   ├── Dockerfile
│   ├── package.json
│   ├── public/
│   │   └── index.html
│   └── src/
│       ├── index.js
│       ├── App.js
│       ├── App.css
│       └── components/
│           ├── Dashboard.js
│           ├── Parcelles.js
│           ├── Arbres.js
│           ├── Interventions.js
│           ├── Recoltes.js
│           ├── Clients.js
│           ├── Ventes.js
│           └── Statistiques.js
└── nginx/
    └── nginx.conf
```

6. Lancer l'application

Option A : Avec le script automatique (Linux/Mac)

```
bash

chmod +x setup.sh
bash setup.sh
```

Option B : Manuellement

```
bash
```

```
docker-compose up --build
```

7. Vérifications finales

☐ Tous les conteneurs Docker sont démarrés

```
bash
```

```
docker-compose ps
```

☐ L'API répond

```
bash
```

```
curl http://localhost:3001/api/health
```

☐ Le frontend est accessible

- Ouvrir <http://localhost:3000> dans le navigateur

☐ Les données de démo sont présentes

```
bash
```

```
curl http://localhost:3001/api/parcelles
```

☐ Navigation fonctionnelle

- Tester tous les onglets du menu

☐ Base de données accessible

```
bash
```

```
docker exec -it truffiere_db psql -U truffiere_user -d truffiere
```

```
\dt # Lister les tables
```

```
\q # Quitter
```

Fonctionnalités implémentées

☒ Gestion de la culture

☒ Création/édition/suppression de parcelles

☒ Inventaire des arbres truffiers

☒ Planning des interventions

☒ Types d'interventions prédéfinis

☒ Gestion de la production

- ☒ Enregistrement des récoltes
- ☒ Suivi du poids et de la qualité
- ☒ Gestion des clients
- ☒ Enregistrement des ventes

☒ **Historique et statistiques**

- ☒ Traçabilité automatique (triggers SQL)
- ☒ Statistiques par parcelle
- ☒ Statistiques par arbre
- ☒ Analyse des ventes
- ☒ Tableaux de bord interactifs

☒ **Architecture technique**

- ☒ Docker Compose pour orchestration
- ☒ PostgreSQL avec PostGIS
- ☒ API REST complète (Express)
- ☒ Interface React moderne
- ☒ Vues SQL pour statistiques
- ☒ Index pour optimisation

Fonctionnalités à développer (optionnel)

☐ **Court terme**

- ☐ Formulaires complets pour tous les composants
- ☐ Validation des données côté frontend
- ☐ Messages d'erreur utilisateur
- ☐ Confirmation de suppression
- ☐ Filtres et recherche

☐ **Moyen terme**

- ☐ Cartographie interactive (Leaflet)
- ☐ Graphiques avancés (Recharts)
- ☐ Export PDF des rapports
- ☐ Upload de photos
- ☐ Authentification utilisateur

☐ **Long terme**

- ☐ Application mobile (React Native)
- ☐ API publique documentée
- ☐ Intégration météo
- ☐ Notifications push
- ☐ Multi-langues



Problèmes courants

Erreur : Port already in use

Solution : Modifier les ports dans `docker-compose.yml`

Erreur : Cannot connect to database

Solution :

```
bash

docker-compose down -v
docker-compose up --build
```

Page blanche sur <http://localhost:3000>

Solution :

1. Vérifier les logs : `docker-compose logs -f frontend`
2. Vérifier que tous les composants sont créés
3. Vider le cache : Ctrl+F5

Module not found

Solution :

```
bash

docker-compose down
rm -rf backend/node_modules frontend/node_modules
docker-compose up --build
```



Support

Consultez les fichiers de documentation :

- **Questions générales** → `README.md`
- **Installation** → `INSTRUCTIONS_DEPLOYMENT.md`
- **Commandes** → `QUICK_REFERENCE.md`
- **Personnalisation** → `CUSTOMIZATION.md`

Pour les logs :

```
bash

docker-compose logs -f
```

 **Prêt !**

Si toutes les cases sont cochées, votre application est prête à l'emploi ! 

Bon travail avec votre gestion de truffière !